TMPA Implementation and Case Report

Zhu Wei - joinwell52 Research Center

2026-08-10 - I0.8 - RC1

Table of Contents

1 TMPA Implementation and Case Report

1.1 TMPA Core S0.6, FCoP, CodeFlowMu V1.6.0, and retained field evidence

Document Version: Draft I0.8

Status: Author-produced implementation and case report

Normative Target: TMPA Core Specification S0.6, commit 8989657e8fde6d2e55d7606ae0adacac14fec760

Product Under Test: CodeFlowMu V1.6.0, implementation commit 62440a526a99d2fe55019d8bb95ad2425569fcb4

Evidence Capture: 2026-08-10, Asia/Shanghai

Formal Evidence Package: tmpa-i0.8-codeflowmu-v1.6.0-s0.6-evidence-20260810.zip

Archive SHA-256: 3c34514089f08f5957d806f900ab31af1cdae94c08f31a5da046f451b5884fe9

Authority Boundary: This report is evidentiary and non-normative. Only the Core Specification defines TMPA requirements.

1.2 Abstract

I0.8 evaluates CodeFlowMu V1.6.0 against all fourteen mandatory criteria in TMPA Core S0.6 using the exact raw-LF bytes fixed by the normative repository commit. The product-level result is **14 PASS, 0 PARTIAL, 0 NOT RUN, and 0 FAIL**. CodeFlowMu's product runner calls its own synchronous GovernanceReader and does not invoke the TMPA Reference Reader. The input bundle digest is sha256:251914ee55922d20c9bd23943a4ff445bccaa5835e1fcc11b8562f3f384243fa.

The engineering upgrade closes the observable S0.6 delta over the I0.7/S0.5 baseline: byte-identical observations retain every contributing source ID; high-risk approval requires a permitted decision-object type, a valid role assignment, an allowed role, an approval decision, and an independent actor when required; and every canonical sort uses locale-independent Unicode code-point order, including a regression over U+E000 and U+10000. CodeFlowMu's internal TMPA suite passes 23/23 tests, Runtime records 1,485 passed / 0 failed / 1 skipped, Shell records 777/777, and the locked FCoP reference implementation records 1,210 passed / 2 skipped.

The evidence archive includes a self-contained public reproducer. A clean npm ci and npm test run verifies the seven official S0.6 byte digests and executes the bundled CodeFlowMu product Reader with 14/14 PASS; no private CodeFlowMu checkout is required for this conformance slice. The strongest supportable claim remains **author-run demonstrated behavior under a fixed bundle**. The result is not independent certification, independent adoption, a proof for arbitrary inputs, or proof that AI hallucinations have been eliminated.

2 1. Scope and Research Questions

I0.8 asks:

Does the CodeFlowMu V1.6.0 product Reader satisfy S0.6 C01–C14 against one exact, fixed input bundle?

Are the three observable S0.6 changes—source provenance retention, complete human-approval authorization, and locale-independent ordering—implemented without regressing the S0.5 behavior demonstrated by I0.7?

Can the conformance slice be rerun without access to the private CodeFlowMu mother repository?

Which evidence remains author-local, and which conclusions remain unsupported?

The unit of judgment is a criterion-bound claim over a fixed bundle. The four evidence levels remain separate:

Level

Meaning in I0.8

Reached?

Specified

Defined normatively by TMPA Core S0.6

Yes

Implemented

A corresponding CodeFlowMu mechanism exists

Yes

Demonstrated

An execution produced inspectable evidence

Yes, for the fixed C01–C14 bundle

Independently Adopted

A separate organization adopted and validated the mechanism

No

3 2. Architecture and Evidence Boundaries

TMPA Architecture Paper

↓ theory guides engineering direction

TMPA Core S0.6

↓ fixes normative object, Reader, and conformance contract

FCoP protocol

↓ supplies the collaboration and evidence protocol

CodeFlowMu V1.6.0 execution and consumption layer

↓ product Reader, Runtime, roles, recovery, and audit

WP-13 and other bounded cases

TMPA theory guides the engineering direction of CodeFlowMu, while Core S0.6 fixes the normative behavior evaluated in this report. FCoP is the reusable collaboration and evidence protocol used by CodeFlowMu, not an application; fcop and fcop-mcp are reference implementations rather than the protocol itself. Consistent with CodeFlowMu's engineering architecture [7], CodeFlowMu is the application execution and consumption layer: it produces coordination facts, runs the Adapter and Reader, projects the governance graph, and lets recovery and governance gates consume the reconstructed result. WP-13 remains a bounded evidence-admission case. XiaoDian AI is retained as author-reported engineering lineage only and is excluded from the evaluated evidence; neither substitutes for the S0.6 product run.

Conceptual dependency and historical formation remain distinct:

CURRENT GUIDANCE: TMPA theory → Core requirements → FCoP protocol
→ CodeFlowMu engineering

HISTORICAL FEEDBACK: XiaoDian practice → early TMPA → FCoP extraction
→ CodeFlowMu implementation

FCoP + CodeFlowMu results → current TMPA formalization

Historical feedback explains how the theory matured; it does not invert the current authority relation. Product behavior may provide evidence or motivate a later revision, but it cannot redefine the current Core.

4 3. Fixed Sources and Evidence Design

Source

Fixed identity

Role in I0.8

Boundary

TMPA Core

S0.6, commit 8989657…

Normative C01–C14 target

Specification, not product evidence

CodeFlowMu

V1.6.0, commit 62440a5…

Product implementation under test

Local implementation commit; not represented as a public release

Public reproducer

29 locked files

Public rerun of the conformance slice

Does not expose or reproduce the entire private product

FCoP reference implementation

commit da79dfe…

Locked dependency baseline

Test result is not proof of the abstract protocol

I0.7

S0.5 / CodeFlowMu V1.4.1

Historical regression baseline

Retained with its exact earlier meaning

WP-13 V3

Previously published package

Evidence-gating field case

Not rerun and not promoted to a conformance proof

The seven official S0.6 inputs—four schemas, canonicalization profile, lifecycle profile, and fixtures—are byte-identical to GitHub. The first submitted archive had Windows checkout-converted CRLF copies; publication preflight rejected it. The final archive uses raw Git blob bytes, reruns the product criteria, and regenerates every dependent digest instead of editing prior result JSON.

5 4. Executed Test Baseline

All primary product and full-suite runs were author-executed against fixed inputs. Skipped tests remain skips.

Track

Result

Exit

Interpretation

S0.6 product C01–C14

14 PASS / 0 FAIL / 0 PARTIAL / 0 NOT RUN

0

Direct CodeFlowMu product Reader execution

CodeFlowMu internal TMPA

23/23

0

Includes explicit S0.5 compatibility and S0.6 regressions

CodeFlowMu Protocol

5 valid + 3 expected-invalid fixtures

0

Product protocol tests, separate from TMPA verdict

Protocol / Runtime / Shell type checks

all passed

0

Static checks only

CodeFlowMu Runtime

1,485 passed, 0 failed, 1 skipped

0

Full author-local Runtime run

CodeFlowMu Shell

777 passed, 0 failed

0

Full author-local Shell run

FCoP locked reference baseline

1,210 passed, 2 skipped

0

Reference implementation, not protocol proof

Public reproducer

npm ci + 14/14 product PASS

0

Self-contained conformance-slice rerun

The final conformance result validates against the S0.6 result schema. It records product_reader_called: true, reference_reader_called: false, implementation codeflowmu/V1.6.0, and evidence level demonstrated.

6 5. C01–C14 Product Results

ID

S0.6 criterion

Verdict

Executed observation

C01

Schema validation

PASS

All four published S0.6 schemas compile; malformed structural and asserted date-time cases are rejected deterministically.

C02

Primary carrier and immutability

PASS

Content-addressed revisions remain attributable, primary-carrier conflicts do not gain authority, and corrections remain separate evidence.

C03

Duplicate identity and provenance

PASS

Different-content duplicates are quarantined; byte-identical observations project one node while retaining every deterministically ordered source_id.

C04

Serial continuity and asynchronous progress

PASS

Stream gaps and duplicate sequences stay explicit, unrelated streams progress, and arrival order does not change output.

C05

Role authority

PASS

Denied and undetermined authority branches remain distinct and cannot mutate reconstructed state.

C06

Lifecycle legality

PASS

Illegal transitions are rejected; missing independent acceptance leaves completion undetermined and prevents advancement.

C07

Separation of duties and human approval

PASS

Wrong approval type, missing assignment, disallowed role, missing approval decision, and self-approval remain pending_human; a valid assigned independent approval clears the gate.

C08

Integrity tampering

PASS

Covered-content tampering emits INTEGRITY_MISMATCH, excludes the object from authoritative nodes, and retains the failed source.

C09

Missing reference

PASS

Missing dependencies and claim evidence remain visible and propagate an undetermined result without inventing facts.

C10

Prohibited cycle

PASS

Only the affected prohibited-cycle subgraph is quarantined; unrelated valid nodes remain usable.

C11

Deterministic reconstruction

PASS

Fixed-source permutations are byte-equivalent; canonical order is Unicode code-point order, including U+E000 before U+10000.

C12

Conflict preservation

PASS

Contradictory reviews remain disputed until an authorized resolver acts; unauthorized decisions remain evidence but cannot clear the conflict.

C13

Recovery

PASS

A fresh Reader reconstructs lifecycle, responsibility, dependencies, failures, recovery actions, and parent-child state consistently.

C14

Terminal-history preservation

PASS

Only the accepted authorized chain reaches archive, while the prior task/report/review/transition history remains reconstructable.

7 6. S0.5 to S0.6 Product Delta

7.1 6.1 C03 — aggregation without provenance loss

I0.7 demonstrated conflict handling for same-ID, different-content candidates. S0.6 adds a second boundary: multiple byte-identical observations must not create duplicate nodes, but aggregation must not erase where those observations came from. V1.6.0 emits one canonical node with the complete, code-point-sorted source_ids list.

7.2 6.2 C07 — approval is an authorized governance object

The S0.5 baseline demonstrated role separation and a high-risk human gate. S0.6 makes the authorization contract directly testable. An approval counts only when its object type is permitted, the actor has a matching Assignment, the role is allowed, the body carries an approval decision, and the actor is independent when the Profile requires it. Four negative branches remain pending rather than being treated as implicit approval.

7.3 6.3 C11 — deterministic means locale-independent

S0.6 removes environmental ambiguity from canonical sorting. V1.6.0 replaces locale-dependent comparison with Unicode code-point ordering throughout object keys, sources, graph traversal, issues, and output collections. The U+E000/U+10000 fixture detects the UTF-16 code-unit ordering mistake that can otherwise pass ordinary ASCII tests.

7.4 6.4 Compatibility boundary

CodeFlowMu's internal suite includes an explicit S0.5/I0.7 compatibility path. I0.7 remains the exact historical S0.5 product result; I0.8 does not rewrite its inputs, criteria, product version, or evidence package.

8 7. Public Reproducer

The formal archive contains a 29-file self-contained reproducer with a package lock, the product Reader, Protocol Validator, Profile, Fixtures, Runner, and exact official inputs. It requires Node.js 22 or newer and public npm registry access:

```
cd public-reproducer
npm ci
npm test
```

The test first verifies all seven upstream SHA-256 values, then calls the bundled CodeFlowMu GovernanceReader. The recorded clean-copy run and publication preflight both produce 14 PASS / 0 FAIL. The reproducer makes the conformance slice publicly inspectable without asserting that the private mother repository or full application is public.

9 8. Retained WP-13 Evidence-Gating Case

WP-13 remains relevant to the engineering interpretation but is not rerun in I0.8. It records a role-separated workflow in which a completion-meaning claim was held until persistent evidence, a formal report, and QA verification existed. A later Gate C decision accepted delivery while activation, push/publication, and archive remained separate decisions.

This case supports evidence gating, staged authority, and same-task recovery. It does not prove hallucination elimination, universal false-claim detection, or S0.6 conformance. The C01–C14 result comes from the V1.6.0 product bundle, not from WP-13.

10 9. Three-Valued Governance Interpretation

TMPA keeps semantic judgment separate from view classification:

Judgment

Typical view

Meaning

valid

authoritative

Required evidence and applicable rules establish the conclusion.

invalid

quarantined / rejected

A deterministic violation excludes the affected evidence or action.

undetermined

partial / disputed / pending_human

Evidence is missing or conflicting, or an authorized human decision is still required.

I0.8 makes the separation observable. A wrong-type or self-issued approval is preserved but cannot satisfy C07. A missing reference in C09 leaves the dependent claim undetermined rather than false

or complete. An unauthorized C12 resolution remains evidence but is invalid as a resolving act. Integrity failure in C08 quarantines covered content while preserving its source record.

11 10. Evidence Integrity and Publication Preflight

The formal archive contains 195 files; 194 payload files are covered by its internal SHA-256 manifest. Its outer SHA-256 is 3c34514089f08f5957d806f900ab31af1cdae94c08f31a5da046f451b5884fe9. Independent publication preflight verified:

ZIP structural integrity and ASCII-only entry names;

strict UTF-8 decoding for every file;

137 JSON files and 11 JSONL records;

194/194 internal SHA-256 entries;

byte identity for all seven official S0.6 inputs;

schema-valid C01–C14 result envelopes;

product Reader invocation without Reference Reader substitution;

a self-contained reproducer result of 14/14 PASS.

The initial 2026-08-09 candidate archive is not published. It contained semantically identical but checkout-converted CRLF copies of the official inputs, an incorrect manifest-count statement, and no self-contained public rerun path. The corrected 2026-08-10 archive replaces it as the sole I0.8 formal package.

12 11. Limitations

The product and full-suite evidence is author-run; no independent organization has certified or adopted the implementation.

The CodeFlowMu implementation commit is local and is not represented as a public release, tag, or complete public source tree.

The public reproducer exposes the conformance slice, not the entire private CodeFlowMu application or its full Runtime/Shell environment.

Runtime retains one skipped test; the FCoP reference implementation retains two migrated-layout historical-example skips.

C11 evaluates a fixed fixture set and its declared permutations; it is not a formal proof over arbitrary graphs or hostile platforms.

C08 demonstrates governance-object integrity handling, not model truthfulness, identity authentication, installer protection, or Byzantine resilience.

Full-suite performance, representative SME burden, comparative baselines, and independent deployment remain open empirical questions.

WP-13 is a bounded governance case, not a hallucination-elimination benchmark.

13 12. Claim Ledger

Claim

I0.8 disposition

TMPA Core S0.6 defines C01–C14

Specified

CodeFlowMu V1.6.0 contains corresponding product mechanisms

Implemented

The fixed product bundle records 14/14 PASS

Demonstrated

The self-contained conformance slice can be publicly rerun

Demonstrated

The complete private CodeFlowMu application is publicly reproducible

Not claimed

The result has been independently certified or adopted

Not demonstrated

WP-13 proves hallucination elimination

Prohibited conclusion

14 13. Engineering Conclusion

I0.8 advances the Implementation Case from an exact S0.5 product result to an exact-input S0.6 product result. CodeFlowMu V1.6.0 passes all fourteen mandatory criteria while directly exposing the S0.6 changes in provenance retention, approval authority, and locale-independent reconstruction. A self-contained reproducer narrows the gap between author-local product evidence and public inspection without misrepresenting the private full application as open or independently validated.

The result strengthens the engineering evidence for TMPA's implementability, not the logical truth of the theory. TMPA theory guides the system design, Core S0.6 fixes the evaluated requirements, FCoP supplies the collaboration and evidence protocol, CodeFlowMu is the application execution

and consumption layer under test, and WP-13 is a bounded field case. Independent adoption and broader empirical evaluation remain future work.

15 Artifact Availability

The formal I0.8 archive is tmpa-i0.8-codeflowmu-v1.6.0-s0.6-evidence-20260810.zip. The adjacent file tmpa-i0.8-codeflowmu-v1.6.0-s0.6-evidence-20260810.zip.sha256 records 3c34514089f08f5957d806f900ab31af1cdae94c08f31a5da046f451b5884fe9.

I0.7 and its V1.4.1/S0.5 archive remain available at their versioned paths. The rejected I0.8 candidate is not part of the public publication set. Git history is the publication history; no parallel paper database has editorial authority.

16 References

[1] TMPA Project. “TMPA Core Specification S0.6,” commit 8989657e8fde6d2e55d7606ae0adacac14fec760. GitHub, 2026.

[2] TMPA Project. “TMPA Architecture Paper A0.9.” GitHub, 2026.

[3] FCoP Project. “FCoP — File-based Coordination Protocol,” reference implementation commit da79dfefd99f597c9e422ce9edec22157f915a21. GitHub, 2026.

[4] CodeFlowMu Project. “CodeFlowMu V1.6.0 S0.6 Product Conformance,” implementation commit 62440a526a99d2fe55019d8bb95ad2425569fcb4, 2026.

[5] TMPA Project. “I0.8 CodeFlowMu V1.6.0 S0.6 Evidence,” package tmpa-i0.8-codeflowmu-v1.6.0-s0.6-evidence-20260810, 2026.

[6] CodeFlowMu Project. “WP-13 Multi-Agent Fact-Check Publication Evidence V3,” 2026.

[7] CodeFlowMu Project. “CodeFlowMu V1.4–V1.5 TMPA × FCoP × Application Unified Architecture,” docs/TMPA-GOVERNANCE.md. GitHub, 2026. https://github.com/joinwell52-AI/codeflowmu/blob/main/docs/TMPA-GOVERNANCE.md.